```
  p.call
  puts "Never get here"
end

meth4     # => 99
```

A lambda behaves more like a free-standing method body: a return simply returns from the block to the caller of the block:

```
def meth5
  p = lambda { return 99 }
  res = p.call
  "The block returned #{res}"
end

meth5     # => "The block returned 99"
```

Because of this, if you use define_method, you'll probably want to pass it a proc created using lambda, not new, because return will work as expected in the former and will generate a LocalJumpError in the latter.

Exceptions

Ruby exceptions are objects of class Exception and its descendents.

Raising Exceptions

The raise method raises an exception:

raise
raise *string*
raise *thing* ‹, *string* ‹, *stack trace*››

The first form reraises the exception in $! or creates a new RuntimeError if $! is nil. The second form creates a new RuntimeError exception, setting its message to the given string. The third form creates an exception object by invoking the method exception on its first argument, setting this exception's message and backtrace to its second and third arguments. Class Exception and objects of class Exception contain a factory method called exception, so an exception class name or instance can be used as the first parameter to raise.

When an exception is raised, Ruby places a reference to the Exception object in the global variable $!.

Handling Exceptions

Exceptions may be handled in the following ways:

- Within the scope of a begin/end block:

begin
  code...
  code...
‹rescue ‹, parm›* ‹, => *var*› ‹, then›
  error handling code... ›*
‹else
  no exception code...›
‹ensure